

git

parte 2: gestión de ramas

Repasamos

Lo que vimos en la parte 1:

- `git config` : nombre, email
- `git init` : ir a una carpeta y comenzar con git
- `git status` : me dice qué ficheros hay para "comitear" y cuáles no
- `git add` : marcar los ficheros para "comitear"
- `git commit` : guardo el estado
- `git log` : muestro el historial de commits

Un extra antes de empezar: `git commit --amend`

```
git commit -m "<msg>"
```

Si te acabas de equivocar en el mensaje del **último** commit, no hace falta deshacer nada: puedes modificarlo directamente.

```
git commit --amend -m "<nuevo_msg>"
```

Solo afecta al **último** commit, y solo deberías usarlo si ese commit todavía no lo ha visto nadie más (no se ha compartido con el resto del equipo).

Comandos git que veremos hoy

- `git branch`
- `git checkout`
- `git merge` (gestión de conflictos)
- `git reset`
- `git revert`

¿Qué es HEAD?

HEAD es un puntero que indica **en qué commit estás situado ahora mismo** — normalmente, el último commit de la rama en la que estás trabajando.

- Cuando haces un commit nuevo, HEAD avanza con él.
- Cuando cambias de rama, HEAD pasa a apuntar al último commit de esa otra rama.
- Lo verás aparecer explícitamente en `git log (HEAD -> main)` y en los conflictos de `git merge`.

Lo necesitas para entender bien los próximos comandos: `branch`, `checkout` y `merge` trabajan todos moviendo o comparando con HEAD.

HEAD en acción

Acabas de hacer un commit en `main` :

```
8b71e02 (HEAD -> main) Commit inicial del portfolio
```

Haces `git checkout -b feature/seccion-contacto` y un commit ahí:

```
9a2c1b0 (HEAD -> feature/seccion-contacto) Añado sección de contacto  
8b71e02 (main) Commit inicial del portfolio
```

`HEAD` se ha movido: ya no apunta a `main` , apunta a la rama nueva.

Vuelves con `git checkout main` :

```
9a2c1b0 (feature/seccion-contacto) Añado sección de contacto  
8b71e02 (HEAD -> main) Commit inicial del portfolio
```

git branch

Permite gestionar las ramas dentro de un repositorio.

Las ramas son como líneas de tiempo o versiones paralelas de un proyecto, donde puedes trabajar de manera aislada sin afectar a la rama principal u otras ramas.

Entendiendo las ramas en git...

Supongamos que tenemos un portfolio personal muy sencillo (`index.html` + `styles.css`).

Después de una revisión, nos hemos dado cuenta de que tenemos que cambiar lo siguiente:

- Arreglar un texto mal escrito en la sección "Sobre mí"
- Añadir una sección nueva "Contacto"
- Renombrar las clases CSS de la sección "Proyectos" (están mal nombradas)

¿Cómo gestionarías esto?

Una posible solución sería...

1. El proyecto actual se llama `main`.
2. Creo una rama nueva a partir de `main` y la llamo `fix/arreglo-sobre-mi`.
3. Trabajo en esa rama y realizo la corrección necesaria.
4. Meto los cambios de `fix/arreglo-sobre-mi` en `main`.

Segundo problema

Ahora `main` ya tiene solucionado el primer problema. Continuamos con el segundo:

1. Creo una nueva rama a partir de `main` y la llamo `feature/seccion-contacto`.
2. Trabajo en esa rama y añado la sección "Contacto".
3. Meto los cambios de `feature/seccion-contacto` en `main`.

Tercer problema

`main` ya tiene solucionado el primer y el segundo problema. Continuamos con el tercero:

1. Creo una nueva rama a partir de `main` y la llamo `feature/renombrar-clases-css`.
2. Trabajo en esa rama y renombro las clases CSS de "Proyectos".
3. Meto los cambios de `feature/renombrar-clases-css` en `main`.

`main` ya tiene todas las correcciones.

Traducción a comandos git

1. `git status` → estoy en la rama **main**.
2. `git branch fix/arreglo-sobre-mi` → creo la rama `fix/arreglo-sobre-mi`.
3. `git checkout fix/arreglo-sobre-mi` → me voy a esa rama.
4. Trabajo en `index.html` y realizo las modificaciones necesarias hasta que esa versión sea correcta.
5. `git checkout main` → vuelvo a la rama `main`.
6. `git merge fix/arreglo-sobre-mi` → le digo a git que fusione `main` con `fix/arreglo-sobre-mi`.

¿Cómo serían el resto de features? Exactamente igual, repitiendo los pasos 2-6 con el nombre de rama correspondiente.

Estrategias de ramificación (workflows)

- **Git Flow** — `main` y `develop` permanentes, con `feature/`, `release/` y `hotfix/` temporales. Pensado para software con versiones instaladas y ciclos de release largos.
- **GitHub Flow** — solo `main` (siempre desplegable) + ramas `feature/` cortas que se fusionan directo vía pull request. Pensado para aplicaciones web con despliegue continuo.
- **GitLab Flow** — término medio: `main` + ramas de entorno (`staging`, `production`) o de release, para tener más control sobre qué está desplegado dónde.
- **Trunk-based development** — casi todo el mundo trabaja directo sobre `main` (el "tronco"), con feature flags.

Buena práctica: nombrar bien las ramas

Fíjate que en todos los ejemplos anteriores las ramas empiezan por `feature/`. Es una convención muy extendida para que, con un vistazo a `git branch`, se sepa qué es cada rama:

- `feature/...` — una funcionalidad nueva (`feature/seccion-contacto`)
- `fix/...` — una corrección de un fallo (`fix/arreglo-sobre-mi`)
- `release/...` — preparación de una versión concreta

No es obligatorio, pero ayuda mucho cuando el equipo (o el repositorio) crece.

Comando `git branch`

- Listar las ramas disponibles:

```
git branch
```

- Crear una nueva rama:

```
git branch <nombre_de_rama>
```

- Eliminar una rama ya fusionada:

```
git branch -d <nombre_de_rama>
```

Si la rama tiene cambios sin fusionar y aun así quieres borrarla: `git branch -D <nombre_de_rama>` (fuerza el borrado).

- Renombrar una rama:

Visualizando las ramas

`git log` a secas no te deja ver bien cómo se relacionan las ramas entre sí. Para eso:

```
git log --graph --oneline --all
```

```
* 3f1e948 (HEAD -> main) Fusiono feature/seccion-contacto
| \
|  * 9a2c1b0 (feature/seccion-contacto) Añado sección de contacto
| /
* 124c35f Arreglo texto de "Sobre mí"
* 8b71e02 Commit inicial del portfolio
```

Muy útil para no perderte cuando ya llevas varias ramas abiertas a la vez.

git checkout

El comando `git checkout` es uno de los más utilizados en Git, ya que permite cambiar entre ramas, restaurar archivos o incluso volver a versiones anteriores de tu proyecto.

Aunque Git ahora recomienda usar `git switch` para cambiar de rama y `git restore` para restaurar archivos, `git checkout` sigue siendo muy popular y es fundamental entender cómo funciona.

git checkout — lo básico

- Cambiar de rama:

```
git checkout feature/seccion-contacto
```

- Crear y cambiar de rama en un solo paso:

```
git checkout -b feature/seccion-contacto
```

- Restaurar un fichero a su última versión confirmada:

```
git checkout index.html
```

- Volver a un commit anterior:

```
git log  
git checkout abcd1234
```

Cuidado: "detached HEAD"

Cuando haces `git checkout <hash_de_commit>` (en vez de un nombre de rama), `HEAD` deja de apuntar a una rama y pasa a apuntar directamente a ese commit. Git te avisará con algo como:

```
Note: switching to 'abcd1234'.  
You are in 'detached HEAD' state...
```

Recuerda: normalmente `HEAD` apunta a una **rama**, y es esa rama la que apunta al commit. Aquí te has saltado ese paso intermedio — `HEAD` apunta al commit directamente, sin ninguna rama de por medio.

Detached HEAD — qué puede pasar

Partimos de este historial:

```
3f1e948 (HEAD -> main) Añado footer  
9a2c1b0 Corrijo estilos  
8b71e02 Commit inicial
```

1. Haces `git checkout 9a2c1b0` (detached HEAD) y, sin darte cuenta, haces un commit de prueba:

```
f4a2d31 (HEAD) Pruebo un cambio rápido  
9a2c1b0 Corrijo estilos  
3f1e948 (main) Añado footer  
8b71e02 Commit inicial
```

Fíjate: `f4a2d31` no tiene ninguna rama a su lado. Solo `HEAD` lo señala.

2. Haces `git checkout main` para seguir trabajando. Git te avisa:

git checkout — usos avanzados

- Recuperar un fichero desde otra rama:

```
git checkout feature/seccion-contacto -- index.html
```

- Crear una rama desde un commit específico:

```
git checkout -b feature/seccion-contacto <hash>
```

Fast-forward vs merge de tres vías

Cuando haces `git merge`, git puede resolverlo de dos formas distintas:

Fast-forward — si `main` no ha cambiado desde que creaste tu rama, git simplemente mueve el puntero de `main` hasta el final de tu rama. No hace falta combinar nada.

```
main      main
|         \
A---B (feature) => A---B (feature, main)
```

Merge de tres vías (recursive) — si `main` sí ha cambiado mientras tanto, git crea un **commit de fusión** nuevo que combina ambos historiales.

```
main:      A---C-----M   (M = commit de fusión)
           /      \
feature:    B-----
```

git merge

El comando `git merge` permite combinar los cambios de dos ramas diferentes en un único historial.

git merge — cómo se usa

1. Cambiamos a la rama a la que deseamos incorporar los cambios:

```
git checkout main
```

2. Le añadimos los cambios de `feature/blah`:

```
git merge feature/blah
```

Posibles salidas:

- `Fast-forward` o `Merge made by the 'recursive' strategy.` → todo ha ido bien.
- `CONFLICT (content): Merge conflict in index.html` → tienes que resolver el conflicto tú mismo.

Resolviendo el conflicto

Abrimos el fichero que tiene conflicto (`index.html`):

```
<<<<<< HEAD
<h1>Login Page</h1>
=====
<h1>Login Portal</h1>
>>>>>> feature/login
```

Traducción:

- Línea 1: te indica que en `main` (`HEAD`) tiene lo siguiente.
- Línea 3: final del contenido de `main` .
- Línea 5: te indica que en `feature/login` tenía lo anterior.

Resolviendo el conflicto — pasos

1. Decidimos con qué parte nos queremos quedar (o combinamos ambas a mano):

```
<<<<<< HEAD
<h1>Login Page</h1>
=====
<h1>Login Portal</h1>
>>>>>> feature/login
```

2. Eliminamos el resto de marcadores, dejando solo el contenido final:

```
<h1>Login Page</h1>
```

3. Confirmamos el cambio:

```
git add .
git commit -m "fix conflict"
```

git reset

Se usa para deshacer cambios **moviendo hacia atrás** el puntero de la rama actual. Tiene tres modos:

- `--soft` : mueve el puntero, pero conserva los cambios en el área de preparación (staging).
- `--mixed` (por defecto): mueve el puntero y también saca los cambios del staging, pero los conserva en tu directorio de trabajo.
- `--hard` : mueve el puntero y **borra** los cambios por completo, sin posibilidad de recuperarlos fácilmente.

El problema: `git reset` reescribe el historial. Si esos commits ya los ha descargado alguien más del equipo, les rompes el historial a todos.

⇒ **Prohibido su uso en la asignatura (y en la vida real, salvo que sepas muy bien lo que haces)**

`git revert`

A diferencia de `git reset`, que modifica el historial de commits, `git revert` **crea un nuevo commit** que deshace los cambios de un commit anterior.

Es útil para mantener el historial intacto mientras se deshacen cambios — es la forma segura de "deshacer" un commit que ya se ha compartido con el resto del equipo.

git revert — ejemplo

```
git revert abc1234
```

- `abc1234` es el hash del commit que deseas deshacer.
- **Salida esperada:** Git crea un nuevo commit que invierte los cambios realizados por el commit `abc1234`, pero sin eliminarlo del historial.

Buenas prácticas de ramas

- No trabajes nunca directamente sobre `main` : crea siempre una rama nueva para cada tarea.
- Una rama = **una tarea concreta**. Evita ramas gigantes que mezclan varias funcionalidades.
- Nombra las ramas de forma descriptiva (`feature/...` , `fix/...`).
- En cuanto fusiones una rama, bórrala (`git branch -d`) para no acumular ramas muertas.
- Antes de hacer `git merge` , actualiza `main` y revisa que no haya conflictos evidentes.

Resumen — chuleta de comandos

Comando	Para qué sirve
<code>git commit --amend -m "..."</code>	Modificar el mensaje del último commit
<code>git branch</code>	Listar las ramas
<code>git branch <rama></code>	Crear una rama nueva
<code>git branch -d / -D <rama></code>	Eliminar una rama (fusionada / a la fuerza)
<code>git branch -m <rama></code>	Renombrar una rama
<code>git checkout <rama></code>	Cambiar de rama
<code>git checkout -b <rama></code>	Crear y cambiar de rama a la vez
<code>git checkout <rama> -- <fichero></code>	Recuperar un fichero de otra rama

¿Dudas?

Próxima sesión: trabajo en remoto con GitHub